

# Sparse QFLPN State Evolution: CSR/SpMV Formulation, Parallel Execution, and a Reproducible Million-Element Benchmark

Adrian Roman

September 8, 2026

## Abstract

Large-scale state evolution in Quantum-Fuzzy Logical Petri Nets (QFLPN) cannot be treated as a dense linear-algebra problem once the operator dimension becomes large. This paper develops a sparse computational formulation based on compressed sparse row (CSR) storage and sparse matrix–vector multiplication (SpMV), and evaluates a recorded million-element benchmark under a fixed sequential/parallel protocol. The computational problem is stated independently of the four-qubit reference construction: the relevant dimension is the sparse operator dimension  $N$ , while the number of quantum subsystems  $q$  remains a separate model parameter. CSR reduces storage and arithmetic from dense  $O(N^2)$  dependence to  $O(N + \text{NNZ})$  storage and  $O(\text{NNZ})$  work per SpMV, subject to the declared sparsity structure. The recorded benchmark uses  $N = 1,000,000$  and  $\text{NNZ} = 2,000,000$ . Under the recorded Java 21.0.12, Ubuntu 24.04.4 LTS, x86\_64 configuration with two parallel threads, the sequential execution time was 8.250 ms and the parallel execution time was 3.139 ms, giving a measured speedup of  $2.6281\times$  and maximum absolute error equal to zero for the compared outputs. The result demonstrates a substantial reduction in execution time for this workload, but it does not establish a universal parallel speedup or justify extrapolation to larger dimensions. The paper therefore separates the measured benchmark from scaling hypotheses and provides a reproducible computational specification for subsequent QFLPN studies.

**Keywords:** QFLPN; sparse linear algebra; compressed sparse row; CSR; SpMV; parallel computing; state evolution; reproducible benchmarking; scalability

## 1 Introduction

The transition from a small, explicitly constructed quantum instance to a large computational experiment changes the dominant scientific problem. In a finite four-qubit construction, the state dimension is  $2^4 = 16$  and dense operators are entirely manageable. At large numerical dimensions, however, dense storage scales quadratically and rapidly becomes incompatible with memory capacity. The computational representation must then exploit structure rather than merely increase hardware resources.

This paper addresses that computational layer for QFLPN. The focus is not the semantic definition of QFLPN and not the reproducibility of a small quantum circuit. Instead, the focus is the numerical evolution of a large state vector under a sparse operator represented in compressed sparse row (CSR) format. The principal operation is sparse matrix–vector multiplication (SpMV), which is the basic kernel in many iterative state-evolution procedures.

The scientific question is precise: under a fixed sparse operator and input vector, can a CSR/SpMV implementation preserve numerical correctness while reducing execution time through parallel

row-wise decomposition, and can the resulting benchmark be specified sufficiently well to distinguish observation from extrapolation?

The paper makes five contributions. First, it separates the sparse numerical dimension  $N$  from the quantum-system size  $q$ , preventing an incorrect identification of a million-element sparse vector with a million-qubit quantum state. Second, it gives an explicit CSR computational contract, including storage arrays, indexing convention, arithmetic operation and complexity. Third, it formulates a correctness proposition connecting CSR/SpMV with the corresponding dense mathematical product. Fourth, it records and analyses the available  $N = 1,000,000$ ,  $\text{NNZ} = 2,000,000$  benchmark under its actual execution configuration. Fifth, it defines the boundary between the measured result and future scaling experiments.

The paper is intentionally distinct from the QFLPN formal-operator article and from the four-qubit reproducibility article. Those works establish semantic and finite-instance foundations. The present contribution concerns sparse computational representation and measured execution behavior. The sparse benchmark is therefore not presented as evidence for the four-qubit quantum model, and the four-qubit construction is not used as a performance proxy.

## 2 Problem Formulation and Scientific Scope

Let the numerical state be represented by a vector

$$x \in \mathbb{R}^N \tag{1}$$

for the benchmark kernel. Let the sparse operator be

$$A \in \mathbb{R}^{N \times N}, \tag{2}$$

with NNZ explicitly stored nonzero entries. The state update considered in the benchmark is

$$y = Ax. \tag{3}$$

The benchmark does not claim that every QFLPN transition has this exact numerical form. Rather, it isolates the sparse linear-algebra kernel that can arise after a QFLPN transition has been represented as an operator acting on a numerical state.

The distinction between  $q$  and  $N$  is essential. For an exact pure quantum state of  $q$  qubits, a dense state vector has dimension  $2^q$ . In the present benchmark,  $N = 1,000,000$  is a numerical workload dimension. It is not a statement that the computation represents one million qubits. The two parameters answer different scientific questions:  $q$  describes the underlying quantum register, whereas  $N$  describes the dimension of the numerical representation used by the sparse kernel.

The scope is limited to sparse matrix–vector execution, correctness comparison, and one recorded large workload. No claim is made about quantum advantage, hardware execution, physical noise, or asymptotic parallel efficiency beyond the measured configuration.

## 3 Sparse Representation

### 3.1 CSR data structure

CSR stores a sparse matrix through three primary arrays. The value array stores the nonzero numerical values, the column-index array identifies the column associated with each stored value, and the row-pointer array identifies the interval belonging to each row.

For row  $i$ , let  $r_i$  and  $r_{i+1}$  be consecutive row pointers. Then

$$y_i = \sum_{k=r_i}^{r_{i+1}-1} a_k x_{c_k}, \quad (4)$$

where  $a_k$  is the stored value and  $c_k$  is its column index. The number of arithmetic contributions to the complete product is proportional to NNZ.

The CSR representation is not merely a storage optimization. It changes the computational contract: the algorithm operates on the explicitly stored nonzero structure rather than on the  $N^2$  entries of a dense matrix. Consequently, correctness depends on the integrity of the row-pointer and column-index arrays as much as on the numerical values.

### 3.2 Complexity

For a matrix with  $N$  rows and NNZ stored entries, CSR storage requires

$$O(N + \text{NNZ}) \quad (5)$$

storage, ignoring fixed-width constants and object overhead. A single SpMV requires

$$O(\text{NNZ}) \quad (6)$$

arithmetic work.

By contrast, a dense representation requires  $O(N^2)$  storage and  $O(N^2)$  arithmetic work for a generic matrix–vector product. Therefore, sparse representation is computationally meaningful only when  $\text{NNZ} \ll N^2$  and the nonzero structure can be exploited without destroying the intended numerical semantics.

**Proposition 1** (CSR correctness). *Assume that every row of  $A$  is represented by exactly the entries encoded between consecutive CSR row pointers, that each stored column index identifies the corresponding matrix column, and that duplicate entries have been resolved according to a fixed representation rule. Then CSR/SpMV computes the same mathematical product  $Ax$  as the corresponding dense matrix representation, up to floating-point evaluation order.*

*Proof.* For row  $i$ , the CSR row interval contains exactly the stored nonzero terms contributing to that row. The implementation evaluates their products with the corresponding components of  $x$  and sums them. Therefore the computed row is the same finite sum defining  $(Ax)_i$ . Repeating this argument for all rows gives the complete matrix–vector product. Floating-point differences can arise only from the arithmetic representation and summation order, not from the CSR mathematical definition itself.  $\square$

## 4 Parallel Row-Wise Execution

CSR SpMV exposes a natural coarse-grained parallelism because each output row can be computed independently when the input vector is read-only during the operation. For a partition of the row interval into disjoint blocks  $R_1, \dots, R_p$ , thread  $j$  computes

$$y_i = \sum_{k=r_i}^{r_{i+1}-1} a_k x_{c_k}, \quad i \in R_j. \quad (7)$$

No write conflict occurs because each output element belongs to exactly one row block.

The measured implementation used a Fork/Join execution model with a threshold of 4096. The threshold determines when recursive partitioning stops and a task performs its assigned work directly. It is therefore part of the benchmark configuration, not an incidental implementation detail.

Parallel execution does not guarantee proportional speedup. The achievable gain is constrained by task creation, scheduling, memory bandwidth, cache behavior, synchronization, load imbalance, and the number of available processing resources. A sparse kernel can therefore become memory-bound even when its arithmetic operation count is small.

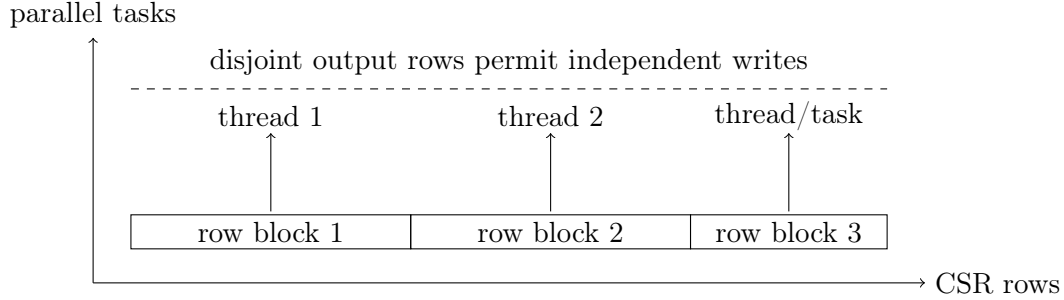


Figure 1: Conceptual row-wise decomposition of CSR SpMV. The diagram specifies the computational independence used by the parallel benchmark; it is not a timing result.

## 5 Benchmark Contract

The benchmark is defined by the recorded configuration rather than by a generic claim of parallel performance. Table 1 contains the parameters available from the recorded experiment.

Table 1: Recorded benchmark configuration.

| Parameter              | Recorded value                          |
|------------------------|-----------------------------------------|
| Workload dimension $N$ | 1,000,000                               |
| Stored entries NNZ     | 2,000,000                               |
| Sequential time        | 8.250 ms                                |
| Parallel time          | 3.139 ms                                |
| Measured speedup       | $2.6281\times$                          |
| Maximum absolute error | 0                                       |
| Parallel threads       | 2                                       |
| Execution model        | Java Fork/Join                          |
| Fork/Join threshold    | 4096                                    |
| Warm-up iterations     | 1                                       |
| Measured iterations    | 2                                       |
| Processor architecture | x86_64                                  |
| Available processors   | 4                                       |
| Runtime                | Java 21.0.12                            |
| Operating system       | Ubuntu 24.04.4 LTS                      |
| Precision              | recorded double-precision configuration |

The benchmark contains only two measured iterations. Consequently, the paper reports the recorded times as measurements, but does not manufacture a standard deviation, confidence interval, or statistical significance claim. A stronger statistical performance study requires substantially more repeated measurements under a fixed environment.

## 6 Measured Results

The measured sequential execution time was 8.250 ms, while the parallel execution time was 3.139 ms. The speedup is

$$S = \frac{T_{\text{seq}}}{T_{\text{par}}} = \frac{8.250}{3.139} = 2.6281. \quad (8)$$

The relative reduction in execution time is

$$R = 1 - \frac{T_{\text{par}}}{T_{\text{seq}}} \approx 0.6195, \quad (9)$$

corresponding to approximately 61.95% lower wall-clock time for the recorded workload.

The maximum absolute error between the compared sequential and parallel outputs was recorded as zero. This is a correctness observation for the recorded comparison, not a proof of correctness for every future partition, matrix structure, or numerical environment.

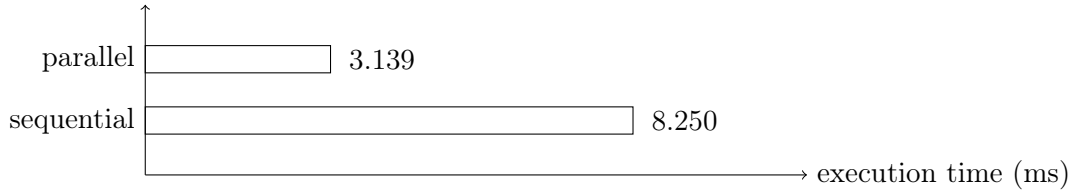


Figure 2: Recorded execution times for the  $N = 1,000,000$ ,  $\text{NNZ} = 2,000,000$  workload. The figure visualizes the measured values and does not extrapolate beyond the experiment.

## 7 Interpretation of the Benchmark

The result establishes three facts within the recorded configuration. First, the parallel implementation completed the specified workload faster than the sequential implementation. Second, the output comparison did not reveal a numerical discrepancy under the recorded test. Third, the observed speedup was below a target of  $3\times$  that had been defined for operational monitoring of this benchmark.

The third observation is important. A speedup of  $2.6281\times$  on two parallel threads is meaningful, but it should not be converted into a claim of scalable parallel efficiency. The experiment does not contain enough repetitions or thread-count points to identify a scaling curve. It also does not isolate memory-bandwidth limitations from scheduling overhead.

The threshold value of 4096 should likewise be interpreted as a configuration parameter rather than an optimality proof. A different threshold can change task granularity, synchronization overhead and locality. Determining an optimal threshold is a separate experiment and belongs to a controlled ablation study.

## 8 Memory and Dense-versus-Sparse Regime

The principal motivation for CSR is the difference between dense and sparse storage. A dense  $N \times N$  matrix contains  $N^2$  entries. At  $N = 1,000,000$ , this is  $10^{12}$  entries. Even if one stores a single real double per entry, the numerical payload alone would require approximately  $8 \times 10^{12}$  bytes, or about 8 TB, before matrix-object and indexing overhead.

For the recorded sparse workload,  $\text{NNZ} = 2 \times 10^6$ . Under an illustrative representation using 64-bit real values, 32-bit column indices and 32-bit row pointers, the raw payload would be approximately

$$8 \text{ NNZ} + 4 \text{ NNZ} + 4(N + 1) \approx 28 \text{ MB.} \quad (10)$$

This is a representation-level estimate, not a measured Java heap footprint. Actual memory consumption depends on object layout, array headers, alignment, JVM implementation and whether 32-bit or 64-bit indices are used.

The contrast is therefore structural: sparse storage makes the million-dimensional workload feasible at the representation level, whereas a generic dense matrix would require orders of magnitude more memory. This comparison is an analytical consequence of the declared dimensions and storage formats, not a measured memory benchmark.

## 9 Reproducibility and Validation Protocol

A future reproduction of the benchmark should preserve the following invariants: the same  $N$ , the same NNZ pattern, the same numerical values and input vector, the same precision, the same row partitioning semantics, the same Fork/Join threshold, and the same comparison rule.

Correctness should be evaluated independently of timing. The primary numerical criterion is

$$\varepsilon_\infty = \max_i |y_i^{(\text{seq})} - y_i^{(\text{par})}|. \quad (11)$$

The recorded value is  $\varepsilon_\infty = 0$ . Additional future diagnostics should include relative error, normwise error, residual against an independently generated reference, and deterministic repeatability under fixed execution conditions.

Performance should be reported with enough repetitions to estimate dispersion. A suitable future protocol would include a longer warm-up, multiple measured repetitions, median and mean times, standard deviation or robust dispersion, confidence intervals where justified, and measurements over several thread counts. Such additions are deliberately not retroactively inserted into the present result.

## 10 Scaling Questions Left Open

The current experiment establishes one large workload, not a scaling law. The next scientifically meaningful experiments are therefore controlled expansions in one variable at a time. The workload dimension  $N$  should be varied while preserving a documented sparsity ratio or a documented structural pattern. The number of threads should be varied while keeping the problem fixed. The task threshold should then be swept under the same workload.

A useful derived quantity is the sparsity ratio

$$s = \frac{\text{NNZ}}{N^2}. \quad (12)$$

For the recorded workload,

$$s = \frac{2 \times 10^6}{10^{12}} = 2 \times 10^{-6}. \quad (13)$$

The corresponding structural density is therefore 0.0002%. This value explains why dense and sparse representations belong to radically different computational regimes for this workload.

The experiment should not be extrapolated automatically to 5 million, 10 million, or 30 million dimensions. Those cases require actual execution and independent resource accounting. In particular, memory pressure, cache behavior and parallel scheduling can change qualitatively with dimension.

## 11 Threats to Validity

The benchmark has several limitations. The first is sample size: only two measured iterations are available. The second is hardware specificity: the result was obtained on one x86\_64 environment with four reported processors and two active parallel threads. The third is implementation specificity: Java 21.0.12 and Fork/Join tasking contribute to the observed timing. The fourth is workload specificity:  $N = 1,000,000$  and  $NNZ = 2,000,000$  do not characterize all sparse QFLPN operators.

The zero maximum absolute error should also be interpreted narrowly. It demonstrates agreement for the compared outputs under the recorded test. It does not imply mathematical exactness in real arithmetic, absence of floating-point differences under other partitions, or correctness under arbitrary sparse matrix construction.

Finally, the dense-memory comparison is analytical rather than empirical. It uses fixed-width arithmetic assumptions to establish the scale of the representation problem; it is not a measurement of a dense Java implementation.

## 12 Position Relative to the QFLPN Article Series

The scientific boundary of this paper is deliberately narrow. The formal article establishes the QFLPN operator semantics and interface contracts. The four-qubit article establishes a reproducible finite reference instance. This paper addresses a different question: how a large numerical state/operator pair can be represented and evolved through sparse linear algebra.

The following distinctions are maintained. The present paper does not re-prove the QFLPN semantic mapping from fuzzy activation to quantum operators. It does not reuse the four-qubit probability distribution as a performance result. It does not claim that CSR is superior to every alternative sparse representation. It does not compare Taylor and Krylov methods; that is a separate numerical-method study. It does not analyse decoherence, tensor networks, or critical-system applications.

This separation is intentional. Each paper has one principal scientific question, one primary result class, and one evidence boundary. Such separation also reduces the risk of redundant publication and uncontrolled text recycling. IEEE explicitly requires authors to identify recycled material, cite the original work, and explain how a new submission differs from prior work when previously published material forms a basis for it [1, 2]. COPE likewise treats duplicate or redundant submission as an ethical problem and recommends explicit originality controls [3].

## 13 Worked CSR Encoding Example

A small symbolic example clarifies the representation without being confused with the million-element benchmark. Consider

$$A = \begin{bmatrix} 4 & 0 & 0 & 2 \\ 0 & 3 & 0 & 0 \\ 1 & 0 & 5 & 0 \\ 0 & 0 & 2 & 6 \end{bmatrix}. \quad (14)$$

Its nonzero values, column indices and row pointers can be written as

$$\text{values} = (4, 2, 3, 1, 5, 2, 6), \quad (15)$$

$$\text{col\_idx} = (0, 3, 1, 0, 2, 2, 3), \quad (16)$$

$$\text{row\_ptr} = (0, 2, 3, 5, 7). \quad (17)$$

The row pointer contains one additional element because it stores the beginning of every row and the terminal position after the last row. This example demonstrates an important implementation invariant: the difference  $r_{i+1} - r_i$  is the number of stored entries assigned to row  $i$ .

For an input vector  $x = (x_0, x_1, x_2, x_3)^T$ , the first and third output components are

$$y_0 = 4x_0 + 2x_3, \quad (18)$$

$$y_2 = x_0 + 5x_2. \quad (19)$$

The same construction is used at large scale; only the number of rows and stored entries changes. The example is included to make the data-structure semantics independently inspectable and is not used as a performance result.

## 14 Arithmetic Intensity and the Memory-Bound Regime

SpMV is structurally different from dense matrix multiplication because the arithmetic performed per stored value is small. For a conventional real-valued SpMV, each stored entry contributes one multiplication and one addition. The arithmetic work is therefore approximately  $2\text{NNZ}$  floating-point operations, subject to the exact counting convention used by a benchmark.

For the recorded workload this gives approximately

$$2\text{NNZ} = 4,000,000 \quad (20)$$

floating-point operations under the two-operation-per-entry convention. The number should be treated as an operation-count estimate rather than a hardware performance measurement.

The associated memory traffic can be substantially larger than the arithmetic demand because the kernel must read values, column indices, row pointers and input-vector elements and write the output vector. The reuse of  $x$  depends on the column pattern, while the sparse values are generally streamed once per operation. Consequently, increasing processor count does not necessarily produce proportional speedup. The measured result of  $2.6281\times$  with two active threads is compatible with a useful parallel gain, but the present experiment does not identify whether the limiting factor is bandwidth, cache locality, scheduling or another implementation effect.

## 15 Load Balance and Task Granularity

The row-wise decomposition is correct whenever output rows are disjoint, but correctness does not imply balanced execution. If one subset of rows contains many more nonzeros than another, equal numbers of rows do not imply equal amounts of work.

Let  $w_i = r_{i+1} - r_i$  denote the number of stored entries in row  $i$ . For a partition into  $p$  tasks, a useful first diagnostic is the ratio

$$B = \frac{\max_j W_j}{\frac{1}{p} \sum_{j=1}^p W_j}, \quad (21)$$

where  $W_j$  is the total number of stored entries assigned to task  $j$ . The ideal value is  $B = 1$ . Larger values indicate potential load imbalance.

This metric is not reported for the recorded benchmark because the available artifact does not provide the complete row-length distribution. It is therefore a defined future diagnostic, not an inferred result. The distinction is important: a parallel speedup can be observed without knowing whether the workload was perfectly balanced, but an explanation of the speedup requires additional measurements.



The threshold of 4096 introduces a second granularity variable. A very small threshold may create excessive task-management overhead; a very large threshold may reduce parallelism and leave available processors idle. The optimal value is therefore workload-dependent and must be determined experimentally rather than assumed.

## 16 Algorithmic Specification

The sequential kernel can be specified independently of the programming language as follows. For each row, the algorithm reads the interval from the row-pointer array, accumulates the products of stored values with the corresponding input entries, and writes one output value. The parallel variant applies the same operation to disjoint row intervals.

Table 2: Algorithmic invariants for sequential and parallel CSR SpMV.

| Invariant            | Requirement                                               |
|----------------------|-----------------------------------------------------------|
| Row ownership        | Each output row is computed exactly once                  |
| Input state          | $x$ is read-only during one SpMV operation                |
| Row-pointer validity | $0 \leq r_i \leq r_{i+1} \leq \text{NNZ}$                 |
| Column validity      | Every $c_k$ belongs to the declared column range          |
| Value alignment      | $a_k$ and $c_k$ describe the same stored entry            |
| Determinism          | The benchmark records the execution and comparison rule   |
| Numerical comparison | Sequential and parallel outputs use the same error metric |
| Task partition       | Parallel tasks own disjoint output-row intervals          |

These invariants define the minimum correctness contract for a sparse implementation. Performance tuning must not change them. In particular, changing the task partition, loop ordering or data layout is acceptable only if the resulting computation remains mathematically equivalent under the stated numerical tolerance.

## 17 Experimental Design for the Next Scaling Campaign

The recorded million-element benchmark is the baseline. A scientifically stronger scaling campaign should vary one factor at a time and retain a fixed reference configuration for all comparisons. The first axis is workload size. The second is thread count. The third is task threshold. The fourth is sparsity pattern.

For workload scaling, candidate dimensions such as  $N = 250,000, 500,000, 1,000,000, 2,000,000$  and larger values can be considered, but only after actual execution and resource checks. If NNZ is increased proportionally, the sparsity ratio should be reported explicitly so that a larger problem is not confused with a denser problem.

For thread scaling, the same matrix and vector should be executed with one, two, three and four threads on the recorded four-processor environment, followed by replication on additional hardware if the study aims at generality. The resulting curve would permit identification of saturation and overhead regimes.

For threshold sensitivity, the value 4096 should be treated as one point in an ablation rather than as an established optimum. A controlled sweep should record runtime, output error and task count while all other variables remain fixed.

For sparsity-pattern sensitivity, at least two structurally different matrices with identical  $N$  and

NNZ should be considered. A uniform row distribution and a skewed row distribution can reveal whether the observed speedup depends on load balance.

## 18 Reporting Rules for a Reproducible Benchmark

A performance result is scientifically stronger when the computational environment and the measurement rule are explicit. The following reporting fields should accompany every future P3 experiment: matrix dimension, number of stored entries, sparsity ratio, value type, index type, input-vector generation rule, initialization cost policy, warm-up count, measured iteration count, thread count, task threshold, processor model, processor count, operating-system version, runtime version, compiler or JIT information where available, and timing source.

Initialization must be separated from steady-state kernel timing unless the research question explicitly concerns end-to-end execution. The same principle applies to memory allocation, data conversion and synchronization. If they are included, they must be included consistently for every compared method.

Correctness and timing should be reported in separate blocks. A fast implementation with an incorrect result is not a successful optimization, while a correct implementation with an unchanged runtime may still be valuable as a reference. The benchmark should therefore preserve both an error metric and a timing metric.

## 19 Evidence Boundary and Future Claims

The available evidence supports a narrow conclusion: under the recorded configuration, the parallel implementation was faster than the sequential implementation for the million-element workload and produced the same compared output. It does not support a claim that CSR/SpMV is optimal for all QFLPN operators, that two threads are optimal, that the threshold 4096 is optimal, or that the speedup will increase or remain constant at larger dimensions.

This evidence boundary is part of the scientific contribution rather than a weakness to be hidden. A grant-quality computational paper benefits from identifying precisely which claims are demonstrated and which experiments are required to extend them. The subsequent numerical-method studies can then use the P3 benchmark as a controlled sparse baseline rather than repeating the same performance claim in different manuscripts.

## 20 Data, Code, and Artifact Availability

The benchmark can be reproduced only if the numerical matrix, input vector and construction procedure are available together with the execution configuration. A timing table without the underlying sparse instance is insufficient for independent replication. The artifact for this study should therefore preserve the CSR arrays or a deterministic generator, the input vector or its generation seed, the sequential and parallel implementations, the comparison routine, and the exact environment metadata.

For the present result, the available evidence records the workload dimensions, execution times, thread count, Fork/Join threshold, warm-up and measured iteration counts, runtime, operating system and processor information. The raw benchmark should be treated as the authoritative source for the reported timing. Any future regenerated measurement must be stored separately and identified by its own configuration so that new observations cannot silently replace the original result.

A useful artifact identifier should encode the problem dimension, sparsity pattern, thread count and threshold. This makes it possible to compare later experiments without relying on filenames or informal descriptions. The same discipline should be used across the QFLPN article series: an experiment belongs to one scientific question and should not be silently reused as evidence for a different claim.

## 21 Comparison Matrix for Future P3 Experiments

The next benchmark campaign should distinguish four comparison questions. The first asks whether sparse representation is necessary for a given dimension. The second asks whether parallel execution improves the same kernel. The third asks whether task granularity affects the result. The fourth asks whether the result is robust to the sparsity pattern.

Table 3: Controlled comparisons proposed for the P3 experimental campaign.

| Comparison              | Variable                | Primary measurements                             |
|-------------------------|-------------------------|--------------------------------------------------|
| Dense vs. sparse        | representation          | memory requirement, runtime, numerical agreement |
| Sequential vs. parallel | thread count            | runtime, speedup, error                          |
| Threshold ablation      | task threshold          | runtime, task count, error                       |
| Pattern sensitivity     | row-length distribution | runtime, load-balance metric, error              |
| Dimension scaling       | $N$ and NNZ             | runtime, memory, speedup, failure point          |

The comparisons must use the same mathematical problem whenever the independent variable is not itself the problem definition. For example, a sequential-versus-parallel comparison is invalid if the two implementations use different sparse matrices or different input vectors. Similarly, a threshold experiment is meaningful only when the matrix, vector, hardware and thread count remain fixed.

## 22 Conclusion

This paper establishes a reproducible sparse-computational formulation for large numerical QFLPN state evolution using CSR storage and SpMV. The mathematical structure gives  $O(N + \text{NNZ})$  storage and  $O(\text{NNZ})$  work per sparse matrix–vector product, provided the operator is represented with the declared sparse structure.

For the recorded million-element workload, the sequential execution time was 8.250 ms and the parallel execution time was 3.139 ms, corresponding to a measured speedup of  $2.6281\times$  and approximately 61.95% lower wall-clock time. The maximum absolute error between the compared outputs was zero. These observations support the correctness and usefulness of the recorded implementation for the tested configuration.

The result does not establish a universal scaling law, an optimal task threshold, or a general parallel advantage. Those claims require dedicated measurements with larger sample sizes, multiple thread counts, workload sizes and controlled ablations. The paper therefore provides a rigorous computational baseline rather than an unsupported extrapolation.

The principal scientific value of the contribution is the separation of sparse representation, correctness, performance measurement and scaling inference into independently testable layers. This

separation creates a clean numerical foundation for subsequent QFLPN studies of matrix functions, convergence, open-system evolution and structured large-scale applications.

## References

- [1] IEEE Author Center, “Avoid Infringement upon IEEE Copyright: Recycling of Text,” IEEE, 2026.
- [2] IEEE Author Center, “Ethical Requirements,” IEEE, 2026.
- [3] Committee on Publication Ethics (COPE), “Handling concurrent and duplicate submissions,” COPE Council, 2024.
- [4] M. Vallero et al., “State of practice: Evaluating GPU performance of state vector and tensor network methods,” *Future Generation Computer Systems*, vol. 174, 107927, 2026.
- [5] A. M. Pastor, J. M. Badia, and M. Castillo, “A community detection-based parallel algorithm for quantum circuit simulation using tensor networks,” *The Journal of Supercomputing*, vol. 81, art. 450, 2025.